

Optimisation de la Couverture Connexe dans les Réseaux de Capteurs (WSN)

Approche Hybride : Glouton Amélioré + VNS + Tabou

Bedis BACCAR Rayen SELMI

Projet Métaheuristique – 2025/2026

Plan de la présentation

- 1 Contexte & Démarche
- 2 Mécanismes d'Optimisation
- 3 Étude Expérimentale
- 4 Analyses de Sensibilité
- 5 Conclusion & Perspectives

Le Problème : Minimiser $|S|$ pour surveiller une zone.

Contraintes :

- **Couverture** :

$$\forall c \in C, \exists s \in S : \text{dist}(c, s) \leq R_{\text{capt}}$$

- **Connexité** : Graphe $(S \cup \{P\})$ connexe avec $\text{dist} \leq R_{\text{com}}$

Notre Démarche Hybride :

- 1 **Construction** : Heuristique Gloutonne
- 2 **Exploration** : VNS (sortir des optimums locaux)
- 3 **Finition** : Tabou (intensification finale)

Pourquoi ce choix ?

- Nature **géométrique** du problème
→ Voisinages visuels intuitifs
- Contrainte de connexité **rigide** → D'autres métaheuristique pourrait casser la structure, VNS la préserve

Heuristique de Construction : Glouton Amélioré

Objectif : Obtenir une solution initiale valide (couverture + connexité) rapidement.

Principe (Expansion Connexe) :

- **Départ** : Le Puits (0,0).
- **Candidats** : Uniquement les cibles à portée de communication ($dist \leq R_{com}$) du réseau existant.
- **Critère de choix** : Maximiser le **gain marginal** (nombre de nouvelles cibles couvertes).

Atout majeur

Cette approche **garantit la connexité par construction**.

Algorithm 1 Glouton Connexe (Simplifié)

```
1:  $S \leftarrow \emptyset, U \leftarrow C$  (Cibles)
2:  $Frontiere \leftarrow Voisins(Puits)$ 
3: while  $U \neq \emptyset$  and  $Frontiere \neq \emptyset$  do
4:   {Sélection du meilleur gain}
5:    $s^* \leftarrow \operatorname{argmax}_{c \in Frontiere} |\{u \in U \mid dist(u, c) \leq R_{capt}\}|$ 
6:    $S \leftarrow S \cup \{s^*\}$ 
7:    $U \leftarrow U \setminus Cibles\_couvertes(s^*)$ 
8:   {Mise à jour de la frontière}
9:    $Frontiere \leftarrow Frontiere \cup Voisins\_com(s^*)$ 
10: end while
11: return  $S$ 
```

Opérateur d'Optimisation (Intensification)

DROP : Supprime un capteur redondant si la solution reste valide (couverture + connexité)

Opérateurs de Perturbation (Diversification)

$k = 1$ **Smart Swap** : Déplacement local dans le R_{com}

$k = 2$ **Add Redundancy** : Ajout temporaire pour connecter îlots

Perturbations fortes

$k = 3$ **Long Jump** : Double Swap (saut plus loin)

$k = 4$ **Ruin & Recreate** : Destruction zone + reconstruction

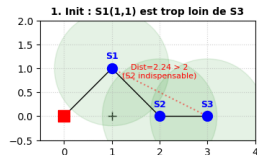
Philosophie

k croissant = intensité de perturbation croissante pour échapper aux bassins d'attraction

Illustration : Mécanisme Swap + Drop

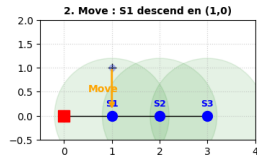
1. Situation Initiale (Optimum Local)

- 3 capteurs : $S_1(1, 1)$, $S_2(2, 0)$, $S_3(3, 0)$
- $dist(S_1, S_3) = \sqrt{5} \approx 2.23 > R_{com}$
- S_2 = relais **indispensable**
- **DROP impossible** → Blocage



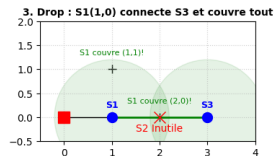
2. Perturbation (Smart Swap)

- S_1 déplacé : $(1, 1) \rightarrow (1, 0)$
- Toujours 3 capteurs, **pas de gain**



3. Optimisation (Drop)

- $dist(S_1, S_3) = 2.0 \leq R_{com} \checkmark$
- S_2 devient **superflu** → Supprimé
- **Bilan : 3 → 2 capteurs**

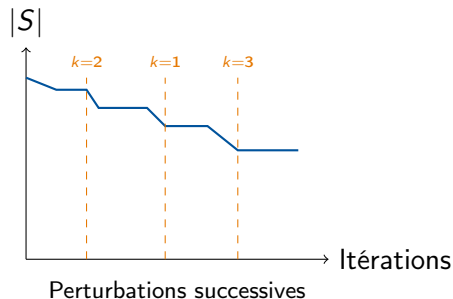


Principe : Alternance dynamique entre *Shaking* et *Local Search*

Algorithme

- 1 Tant que ça s'améliore → Rester à $k = 1$
- 2 Si stagnation → Augmenter k
($1 \rightarrow 2 \rightarrow 3 \rightarrow 4$)
- 3 Si amélioration après secousse → Retour à $k = 1$

Avantage : Adaptation **automatique** de l'intensité à la difficulté du relief local



Clé du succès

Ne jamais rester bloqué : secouer de plus en plus fort si nécessaire

Rôle : Intervenir en “finisseur” après la VNS

Fonctionnement

- **Interdiction** : Ne pas ré-ajouter/bouger un capteur récemment modifié
- **Mémoire** (τ) : Liste taboue (tenure = 12 itérations)
- **Aspiration** : Lever l'interdit si amélioration globale

Pourquoi l'ajouter ?

- VNS = **stochastique** (aléatoire)
- Tabou = **déterministe**, force l'exploration de zones voisines
- Évite le **cyclage** en fin d'optimisation

Calibration

$\tau \approx 10\text{-}15\%$ de $|S_{Heur}|$

Instances de test

Grilles Tronquées :

- 10×10 à 30×30 (87 à 800 cibles)
- Structure régulière

Points Aléatoires :

- 150 à 1600 cibles
- Distribution irrégulière

Configurations testées

R_{capt}/R_{com}	Type
1/1	Rigide
1/2	Souple
2/2	Large couverture
2/3	Très souple

Paramètres clés

VNS : $k_{max} = 4$, $T_{max} \approx 60-260s$

Tabou : Tenure $\tau = 12$

Résultats Globaux – Configuration 1/2

Instance	Cibles	$ S _{Glout}$	$ S _{VNS}$	$ S _{Tabou}$	Gain	Temps
Grille 10×10	90	32	29	29	+9.4%	0.4s
Grille 30×30	669	208	201	201	+3.4%	72s
Aléatoire 900	900	88	81	79	+10.2%	263s
Aléatoire 1600	1600	119	111	105	+11.8%	541s
Moyenne	—	—	—	—	+8.7%	—

Décomposition (Aléatoire 1600)

- Nettoyage initial : **-22.7%** (nettoyage du glouton)
- VNS : **-6.7%** (optimisation structurelle)
- Tabou : **-5.4%** (raffinement final)

Validité

100% des solutions finales respectent couverture + connexité

Config	Gain moyen	Caractère
1/1	~ 5%	Rigide
1/2	> 11%	Souple
2/2	~ 7%	Large
2/3	> 14%	Très souple

Constat clé

Les gains sont **fortement corrélés** à la souplesse topologique

Explication “Physique”

1/1 (Rigide) :

$R_{capt} \approx R_{com} \Rightarrow$ Bouger un capteur pour mieux couvrir **casse souvent la connexion**

1/2 (Souple) :

$R_{com} > R_{capt} \Rightarrow$ marge de manœuvre permet de **reconfigurer en profondeur** sans perdre la connexité, rayon de recherche étendue.

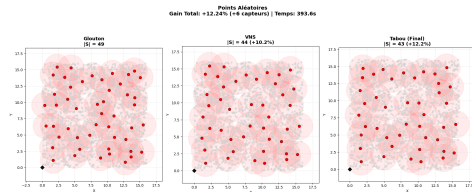
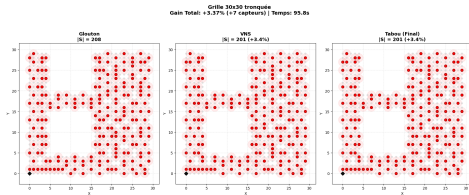
Analyse : Grilles vs Distributions Aléatoires

Sur les Grilles

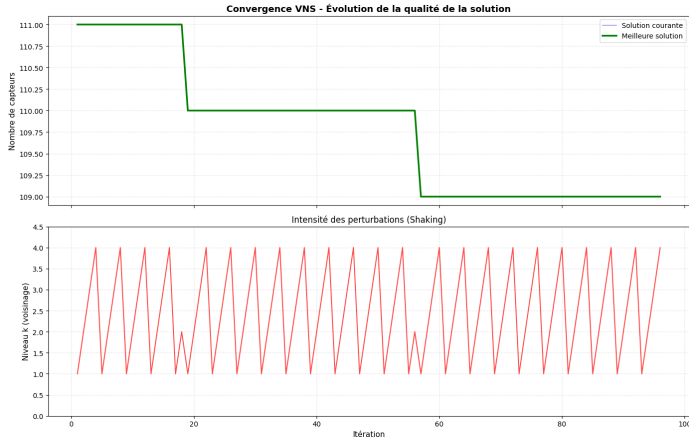
- Gains modestes : $< 5\%$
- Cause : Les **symétries** créent des optimums locaux équivalents
- Difficile de “casser” une structure parfaite

Sur l'Aléatoire

- Gains élevés : $> 10\%$
- Cause : L'**irrégularité** crée des zones denses inutiles
- VNS/Tabou nettoient efficacement



Annexe : Dynamique de Convergence VNS



Corrélation entre intensité du Shaking (niveau k) et amélioration de la solution

Test : Comparaison $\tau = 5$ vs $\tau = 12$ sur Aléatoire 1600

τ	Entrée	Sortie	Drops
12	111	105	6
5	109	107	2

Observation

Malgré un meilleur départ (109 vs 111), $\tau = 5$ produit une solution **inférieure** (107 vs 105)

Conclusion

- Tenure trop courte $\Rightarrow$ **Cyclage**
- L'algorithme "oublie" trop vite
- Ne s'éloigne pas assez des optimums locaux

Règle empirique

$\tau \approx 10\text{-}15\%$ de $|S_{Heur}|$

Points Forts

- **Robustesse** : 100% solutions valides
- **Complémentarité** : VNS (exploration) + Tabou (intensification)

Limites

- **Dépendance structurelle** : Performance liée à la souplesse du réseau (R_{com})
- **Calibration** : Tenure taboue statique & sensible à l'instance

	Grilles (1/1)	Grilles (1/2)	Aléatoire (1/2)	Aléatoire (2/3)
Gain moyen	2-3%	3-5%	10-12%	15-25%

Voisinages Structurels

Développer des mouvements détectant des **motifs géométriques** pour briser les symétries

Hybridation Probabiliste

Intégrer du **Recuit Simulé** dans la VNS pour accepter des dégradations plus fortes et échapper aux optimums profonds

Parallélisation

Évaluer les voisins sur **GPU/Multi-cœur** pour les instances > 5000 cibles

Heuristiques métier

- Éviter capteurs en bord de zone pour grille entière.
- Intégration d'obstacles/relief

Annexe : Tableau Récapitulatif Complet

Instance	Config	$ S _{Glout}$	$ S _{VNS}$	$ S _{Tabou}$	Gain	Temps
Grille 10×10	1/2	32	29	29	+9.4%	0.4s
Grille 10×10	2/2	17	17	17	+0.0%	0.2s
Grille 30×30	1/2	208	201	201	+3.4%	72s
Grille 30×30	2/2	120	114	114	+5.0%	53s
Aléatoire 900	1/2	88	81	79	+10.2%	263s
Aléatoire 900	2/2	36	34	34	+5.6%	111s
Aléatoire 1600	1/2	119	111	105	+11.8%	541s
Aléatoire 1600	2/2	49	44	43	+12.2%	394s

Merci de votre attention

